2 Hello, Spring Security

This chapter covers

- Creating your first project with Spring Security
- Designing simple functionalities using the basic components for authentication and authorization
- The underlying concept and how to use it in a given project
- Applying the basic contracts and understanding how they are correlated
- Writing custom implementations for primary responsibilities
- Overriding Spring Boot's default configurations for Spring Security

Spring Boot appeared as an evolutionary stage for application development with the Spring Framework. Replacing the need to write all the configurations, Spring Boot comes with some preconfigured, so you can override only the configurations that don't match your implementations. We also call this approach *convention-over-configuration*. Spring Boot is no longer a new concept, and today, we enjoy writing applications using its third version.

Before Spring Boot, developers used to write dozens of lines of code repeatedly for all the apps they had to create. This situation was less visible in the past when most architectures were developed monolithically. With a monolithic architecture, you only had to write such configurations once at the beginning, and you rarely needed to touch them afterward. With the evolution of service-oriented software architectures, we started to feel the pain of boilerplate code that we had to write when configuring each service. If you find it amusing, you can check out chapter 3 from *Spring in Practice* by Willie Wheeler with Joshua White (Manning, 2013). This chapter describes writing a web application with Spring 3. In this way, you'll understand how many configurations you had to write for one small one-page web application. The chapter is available at **http://mng.bz/46la**.

For this reason, with the development of recent apps, especially those for microservices, Spring Boot became increasingly popular. Spring Boot provides autoconfiguration for your project and shortens the time required for the setup. We can say it comes with the appropriate philosophy for today's software development.

In this chapter, we'll start with our first application that uses Spring Security. For the apps that you develop with the Spring Framework, Spring Security is an excellent choice for implementing application-level security. We'll use Spring Boot and discuss the defaults configured by convention, with a brief introduction to overriding these defaults. Considering the default configurations provides an excellent introduction to Spring Security, one that also illustrates the concept of authentication.

Once we get started with the first project, we'll discuss various options for authentication in more detail. In chapters 3 through 6, we'll continue with more specific configurations for each of the different responsibilities that you'll see in this first example. You'll also see different ways to apply those configurations, depending on architectural styles. The steps we'll discuss in the current chapter are the following:

1. Create a project with only Spring Security and web dependencies to see how it behaves if no configuration is added. In this way, you'll understand what you should expect from the default configuration for authentication and authorization.

2. Change the project to add functionality for user management by overriding the defaults to define custom users and passwords.
3. After observing that the application authenticates all the endpoints by default, learn that this can be customized as well.
4. Apply different styles for the same configurations to understand best practices.

2.1 Starting your first project

Let's create the first project so that we have something as our first example. This project is a small web application exposing a REST endpoint. You'll see how, without doing much, Spring Security secures this endpoint using HTTP Basic authentication. HTTP Basic is a way in which a web app authenticates a user by means of a set of credentials (username and password) that the app gets in the header of the HTTP request.

NOTE With the default configuration, the app has two different authentication mechanisms in place: HTTP Basic and Form Login. However, I decided to take the example step by step and discuss the Form Login in later chapters. But if you try accessing the URL in a browser, you'll find that your app implements a nice form for user authentication and doesn't display an ugly HTTP Basic box for this reason. I don't want you to be confused in case you decide to experiment with a browser, but we'll focus on this in the section on HTTP Basic.

Just by creating the project and adding the correct dependencies, Spring Boot applies default configurations, including a username and a password, when you start the application.

NOTE You have various alternatives to create Spring Boot projects. Some development environments offer the possibility of creating the project directly. For more details, I recommend Mark Heckler's *Spring Boot: Up and Running* (O'Reilly Media, 2021) and *Spring Boot in Practice* (Manning, 2022) by Somnath Musib or even *Spring Start Here* (Manning, 2021), another book I wrote.

The examples in this book refer to the book's companion source code. With each example, I also specify the dependencies that you need to add to your pom.xml file. You can, and I recommend that you do, download the projects provided with the book and the source code available at **https://www.manning.com/downloads/2105**. The projects will help you if you get stuck with something. You can also use these to validate your final solutions.

NOTE The examples in this book are not dependent on the build tool you choose. You can use either Maven or Gradle. To be consistent, I built all the examples with Maven.

The first project is also the smallest one. It's a simple application exposing a REST endpoint that you can call and then receive a response, as described in figure 2.1. This project is enough for you to learn the first steps when developing an application using Spring Security and Spring Boot. It presents the basics of the Spring Security architecture for authentication and authorization.



Figure 2.1 Our initial application employs HTTP Basic for user authentication and authorization when accessing an endpoint. It offers a REST endpoint at a specified route (/hello). Upon a successful request, it issues an HTTP 200 status message along with a response body. This instance illustrates the default authentication and authorization mechanisms set up by Spring Security.

We begin learning Spring Security by creating an empty project and naming it ssia-ch2-ex1. (You'll also find this example with the same name in other projects provided.) The only dependencies you need to write for our first project are `spring-boot-starter-web` and `spring-boot-starter-security`, as shown in listing 2.1. After creating the project, make sure that you add these dependencies to your pom.xml file. The primary purpose of working on this project is to see the behavior of a default-configured application with Spring Security. We also want to understand which components are part of this default configuration, as well as their purpose.

Listing 2.1 Spring Security dependencies for our first web app

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security</artifactId>
</dependency>
```

We could directly start the application now. Spring Boot applies the default configuration of the Spring context for us based on which dependencies we add to the project. However, we won't be able to learn much about security if we don't have at least one endpoint that's secured. Let's create a simple endpoint and call it to see what happens. For this, we add a class to the empty project, and we name this class `HelloController`. To do that, we add the class to a package called `controllers` somewhere in the main namespace of the Spring Boot project.

NOTE Spring Boot scans for components only in the package (and its subpackages) that contains the class annotated with `@SpringBootApplication`. If you annotate classes with any of the stereotype components in Spring outside of the main package, you must explicitly declare the location using the `@ComponentScan` annotation.

In the following listing, the `HelloController` class defines a REST controller and a REST endpoint for our example.

Listing 2.2 The `HelloController` class and a REST endpoint

```
@RestController
public class HelloController {

  @GetMapping("/hello")
  public String hello() {
    return "Hello!";
  }
}
```

The `@RestController` annotation registers the bean in the context and tells Spring that the application uses this instance as a web controller. In addition, the annotation specifies that the application has to set the response body of the HTTP response from the method's return value. The `@GetMapping` annotation maps the `/hello` path to the implemented method through a GET request. Once you run the application, besides the other lines in the console, you should see something that looks like

```
Using generated security password: 93a01cf0-794b-4b98-86ef-54860f36f7f3
```

Each time you run the application, it generates a new password and prints this password in the console, as presented in the previous code snippet. You must use this password to call any of the application's endpoints with HTTP Basic authentication. First, let's try to call the endpoint without using the `Authorization` header:

```
curl http://localhost:8080/hello
```

NOTE In this book, we use cURL to call the endpoints in all the examples. I consider cURL to be the most readable solution. But if you prefer, you can use a tool of your choice. For example, you might want to have a more comfortable graphical interface. In this case, Postman, Insomnia, or Bruno are excellent choices. If the operating system you use does not have any of these tools installed, you probably need to install them yourself.

And the response to the call is

```
{
 "status":401,
 "error":"Unauthorized",
 "message":"Unauthorized",
 "path":"/hello"
}
```

The response status is HTTP 401 Unauthorized. We expected this result, as we didn't use the proper credentials for authentication. By default, Spring Security expects the default username `(user)` with the provided password (in my case, the one starting with `93a01`). Let's try it again but now with the proper credentials:

```
curl -u user:93a01cf0-794b-4b98-86ef-54860f36f7f3 http://localhost:8080/hello
```

The response to the call is

```
Hello!
```

NOTE The HTTP 401 Unauthorized status code is a bit ambiguous. Usually, it's used to represent a failed authentication rather than an authorization. Developers employ it in the design of the application for cases such as missing or incorrect credentials. For a failed authorization, we'd probably use the 403 Forbidden status. Generally, an HTTP 403 means that the server identified the caller of the request, but they don't have the needed privileges for the call that they are trying to make.

Once we send the correct credentials, you can see in the body of the response precisely what the `HelloController` method we defined earlier returns.

Calling the endpoint with HTTP Basic authentication

With cURL, you can set the HTTP basic username and password with the `-u` flag. Behind the scenes, cURL encodes the string `<username>:<password>` in Base64 and sends it as the value of the `Authorization` header prefixed with the string `Basic`. And with cURL, it's probably easier for you to use the `-u` flag. But it's also essential to know what the real request looks like. So let's give it a try and manually create the `Authorization` header.

In the first step, take the `<username>:<password>` string and encode it with `Base64`. When our application sends the call, we need to know how to form the correct value for the `Authorization` header. You do this using

the Base64 tool in a Linux console. You could also find a web page that encodes strings in Base64, like https://www.base64encode.org. This snippet shows the command in a Linux or a Git Bash console (the `-n` parameter means no trailing new line should be added):

```
echo -n user:93a01cf0-794b-4b98-86ef-54860f36f7f3 | base64
```

Running this command returns the following Base64-encoded string:

```
dXNlcjo5M2EwMWNmMC03OTRiLTRiOTgtODZlZi01NDg2MGYzNmY3ZjM=
```

You can now use the Base64-encoded value as the value of the `Authorization` header for the call. This call should generate the same result as the one using the `-u` option:

```
curl -H "Authorization: Basic dXNlcjo5M2EwMWNmMC03OTRiLTRiOTgtODZlZi01NDg2MGYzNmY3ZjM="  localhost:8080/hello
```

The result of the call is

```
Hello!
```

There are no significant security configurations to discuss with a default project. We mainly use the default configurations to prove that the correct dependencies are in place. It does little for authentication and authorization. This implementation isn't something we want to see in a production-ready application. But the default project is an excellent example that you can use for a start.

With this first example working, at least we know that Spring Security is in place. The next step is to change the configurations so that these apply to our project requirements. First, we'll go deeper with what Spring Boot configures in terms of Spring Security, and then we'll see how we can override the configurations.

2.2 The big picture of Spring Security class design

In this section, we discuss the main actors in the overall architecture that take part in the process of authentication and authorization. You need to know this aspect because you'll have to override these preconfigured components to fit the needs of your application. I'll start by describing how Spring Security architecture for authentication and authorization works, and then we'll apply that to the projects in this chapter. It would be too much to discuss all of these at once, so to minimize your learning efforts in this chapter, I'll discuss the high-level picture for each component. You'll learn details about each in the following chapters.

In section 2.1, you saw some logic executing for authentication and authorization. We had a default user, and we got a random password each time we started the application. We were able to use this default user and password to call an endpoint. But where is all this logic implemented? As you probably know already, Spring Boot sets up some components for you, depending on what dependencies you use (i.e., the convention-over-configuration that we discussed at the beginning of this chapter).

Figure 2.2 shows the big picture of the main actors (components) in the Spring Security architecture and the relationships among them. These components have a preconfigured implementation in the first project. In this chapter, I demonstrate what Spring Boot configures in your application in terms of Spring Security. We'll also discuss the relationships among the entities that are part of the authentication flow presented.

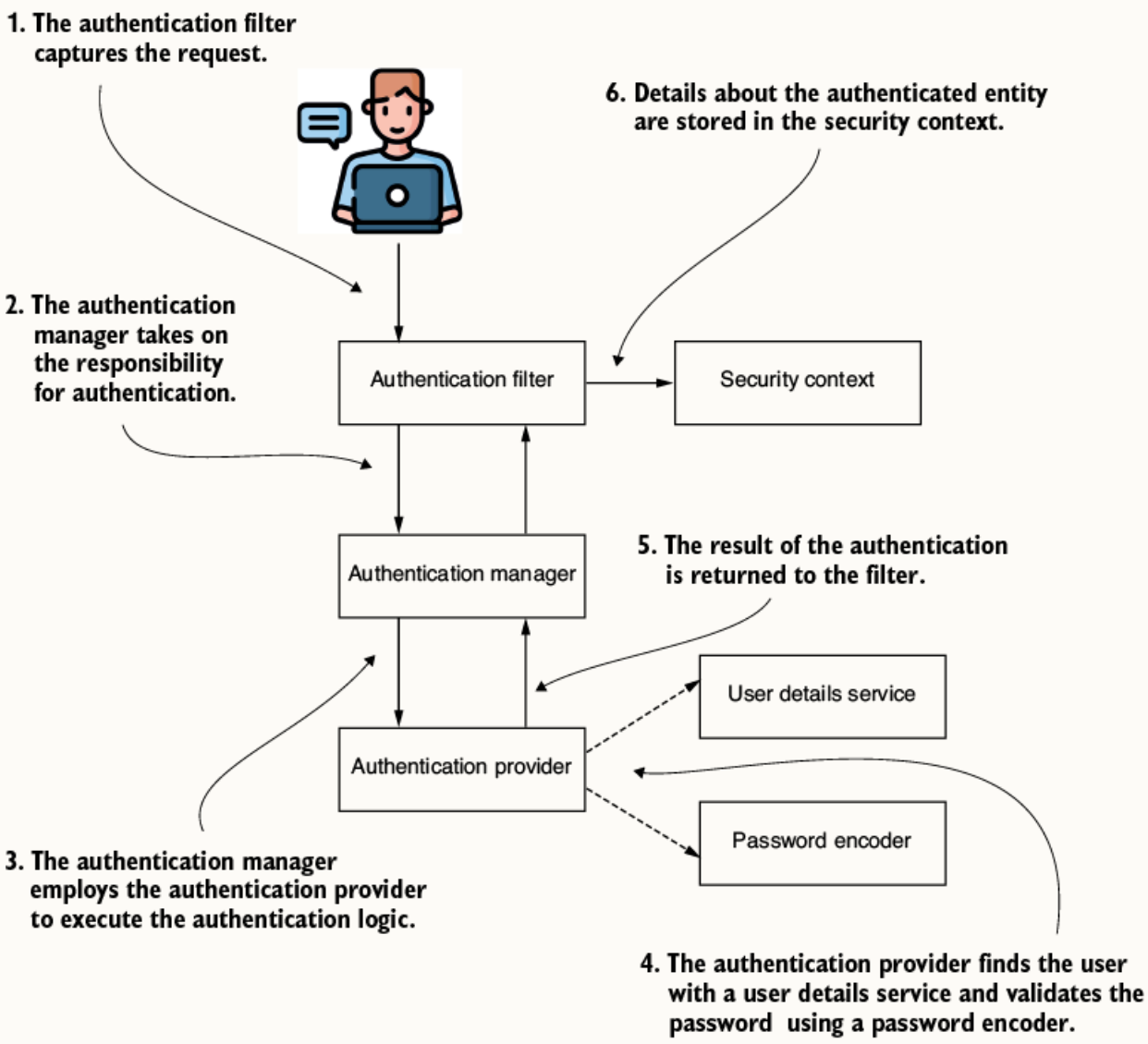


Figure 2.2 The core elements involved in Spring Security's authentication process and their interconnections are the focus here. This framework forms the essential structure for executing authentication using Spring Security. Throughout the book, we will frequently reference this architecture while examining various authentication and authorization strategies.

Figure 2.2 shows that

1. The authentication filter delegates the authentication request to the authentication manager, and based on the response, it configures the security context.
2. The authentication manager uses the authentication provider to process authentication.
3. The authentication provider implements the authentication logic.
4. The user details service implements user management responsibility, which the authentication provider uses in the authentication logic.
5. The password encoder implements password management, which the authentication provider uses in the authentication logic.
6. The security context keeps the authentication data after the authentication process. The security context will hold the data until the action ends. Usually, in a thread-per-request app, that means until the app sends a response back to the client.

In the following paragraphs, I'll discuss these autoconfigured beans:

- `UserDetailsService`
- `PasswordEncoder`

An object that implements a `UserDetailsService` interface with Spring Security manages the details about users. Until now, we've used the default implementation provided by Spring Boot. This implementation only registers

the default credentials in the internal memory of the application. These default credentials are "user" with a default password that's a universally unique identifier (UUID). The default password is generated randomly when the Spring context is loaded (at the app startup). At this time, the application writes the password to the console where you can see it. Thus, you can use it in the example we just worked on in this chapter.

This default implementation serves only as a proof of concept and allows us to see that the dependency is in place. The implementation stores the credentials in-memory—the application doesn't persist the credentials. This approach is suitable for examples or proof of concepts, but you should avoid it in a production-ready application.

Then we have the `PasswordEncoder`. The `PasswordEncoder` does two things:

- Encodes a password (usually using an encryption or a hashing algorithm)
- Verifies if the password matches an existing encoding

Even if it's not as obvious as the `UserDetailsService` object, the `PasswordEncoder` is mandatory for the Basic authentication flow. The simplest implementation manages the passwords in plain text and doesn't encode these. We'll discuss in more detail the implementation of this object in chapter 4. For now, you should be aware that a `PasswordEncoder` exists together with the default `UserDetailsService`. When we replace the default implementation of the `UserDetailsService`, we must also specify a `PasswordEncoder`.

Spring Boot also chooses an authentication method when configuring the defaults: HTTP Basic access authentication. It's the most straightforward access authentication method. Basic authentication only requires the client to send a username and a password through the HTTP `Authorization` header. In the value of the header, the client attaches the prefix `Basic`, followed by the Base64 encoding of the string that contains the username and password, separated by a colon (:).

NOTE HTTP Basic authentication doesn't offer confidentiality of the credentials. Base64 is only an encoding method for the convenience of the transfer; it's not an encryption or hashing method. While in transit, if intercepted, anyone can see the credentials. Generally, we don't use the HTTP Basic authentication without at least HTTPS for confidentiality. You can read the detailed definition of HTTP Basic in RFC 7617 (**https://tools.ietf.org/html/rfc7617**).

The `AuthenticationProvider` defines the authentication logic, delegating the user and password management. A default implementation of an `AuthenticationProvider` uses the default implementations provided for the `UserDetailsService` and the `PasswordEncoder`. Implicitly, your application secures all the endpoints. Therefore, the only thing that we need to do for our example is to add the endpoint. Also, there's only one user who can access all endpoints, so we can say that there isn't much to do about authorization in this case.

HTTP vs. HTTPS

You might have observed that in the examples presented, I only use HTTP. In practice, however, your applications communicate only over HTTPS. For the examples we discuss in this book, the configurations related to Spring Security aren't different, whether we use HTTP or HTTPS. We won't configure HTTPS for the endpoints in the examples so that you can focus on the examples related to Spring Security. But if you want, you can enable HTTPS for any of the endpoints, as presented in this sidebar.

There are several patterns for configuring HTTPS in a system. In some cases, developers configure HTTPS at the application level; in others, they might use a service mesh, or they could choose to set HTTPS at the infrastructure level. With Spring Boot, you can easily enable HTTPS at the application level, as you'll learn in the next example in this sidebar.

In any of these configuration scenarios, you need a certificate signed by a certification authority (CA). Using this certificate, the client that calls the endpoint knows whether the response comes from the authentication server and that nobody intercepted the communication. You can buy such a certificate if you need it. If you only need to configure HTTPS to test your application, you can generate a self-signed certificate using a tool such as OpenSSL (**https://www.openssl.org/**). Let's generate our self-signed certificate and then configure it in the project:

```
openssl req -newkey rsa:2048 -x509 -keyout key.pem -out cert.pem -days 365
```

After running the `openssl` command in a terminal, you'll be asked for a password and details about your CA. Because it is only a self-signed certificate for a test, you can input any data there; just make sure to remember the password. The command outputs two files: key.pem (the private key) and cert.pem (a public certificate). We'll use these files further to generate our self-signed certificate for enabling HTTPS. In most cases, the certificate is the Public Key Cryptography Standards #12 (PKCS12). Less frequently, we use a Java KeyStore (JKS) format. Let's continue our example with a PKCS12 format (for an excellent discussion on cryptography, I recommend *Real-World Cryptography* by David Wong [Manning, 2020]):

```
openssl pkcs12 -export -in cert.pem -inkey key.pem -out certificate.p12 -name "certificate"
```

The second command we use receives as input the two files generated by the first command and outputs the self-signed certificate.

Note that if you run these commands in a Bash shell on a Windows system, you might need to add `winpty` before it:

```
winpty openssl req -newkey rsa:2048 -x509 -keyout key.pem -out cert.pem -days 365
winpty openssl pkcs12 -export -in cert.pem -inkey key.pem -out certificate.p12 -name "certificate"
```

Finally, having the self-signed certificate, you can configure HTTPS for your endpoints. Copy the certificate.p12 file into the resources folder of the Spring Boot project, and add the following lines to your application.properties file:

```
server.ssl.key-store-type=PKCS12
server.ssl.key-store=classpath:certificate.p12
server.ssl.key-store-password=12345          ①
```

① The value of the password is the one you specified when running the second command to generate the PKCS12 certificate file.

The password (in my case, `12345`) was requested in the prompt after running the command for generating the certificate. This is why you don't see it in the command. Now let's add a test endpoint to our application and then call it using HTTPS:

```
@RestController
public class HelloController {

  @GetMapping("/hello")
  public String hello() {
    return "Hello!";
```

```
  }
}
```

If you use a self-signed certificate, you should configure the tool you use to make the endpoint call so that it skips testing the authenticity of the certificate. If the tool tests the authenticity of the certificate, it won't recognize it as being authentic, and the call won't work. With cURL, you can use the `-k` option to skip testing the authenticity of the certificate:

```
curl -k -u user:93a01cf0-794b-4b98-86ef-54860f36f7f3  https://localhost:8080/hello
```

The response to the call is

```
Hello!
```

Remember that even if you use HTTPS, the communication between components of your system isn't bulletproof. Many times, I've heard people say, "I'm not encrypting this anymore. I'll use HTTPS!" While helpful in protecting communication, HTTPS is just one of the bricks of the security wall of a system. Always treat the security of your system with responsibility and take care of all the layers involved.

2.3 Overriding default configurations

Now that you know the defaults of your first project, it's time to see how you can replace them. You need to understand the options you have for overriding the default components because this is the way you plug in your custom implementations and apply security as it fits your application. And as you'll learn in this section, the development process is also about how you write configurations to keep your applications highly maintainable. With the projects we'll work on, you'll often find multiple ways to override a configuration. This flexibility can create confusion. I frequently see a mix of different styles of configuring different parts of Spring Security in the same application, which is undesirable. Thus, this flexibility comes with a caution. You need to learn how to choose from these, so this section is also about knowing what your options are.

In some cases, developers choose to use beans in the Spring context for the configuration. In other cases, they override various methods for the same purpose. The speed with which the Spring ecosystem evolved is probably one of the main factors that generated these multiple approaches. Configuring a project with a mix of styles is not desirable, as it makes the code difficult to understand and affects the maintainability of the application. Knowing your options and how to use them is a valuable skill, and it helps you better understand how you should configure application-level security in a project.

In this section, you'll learn how to configure a `UserDetailsService` and a `PasswordEncoder`. These two components usually take part in authentication, and most applications customize them depending on their requirements. While we'll discuss details about customizing them in chapters 3 and 4, it's essential to see how to plug in a custom implementation. The implementations we use in this chapter are all provided by Spring Security.

2.3.1 Customizing user details management

The first component we talked about in this chapter was `UserDetailsService`. As you saw, the application uses this component in the authentication process. In this section, you'll learn to define a custom bean of type `UserDetailsService`. We'll do this to override the default one configured by Spring Boot. As you'll see in more detail in chapter 3, you have the option to create your own implementation or to use a predefined one provided by

Spring Security. In this chapter, we aren't going to detail the implementations provided by Spring Security or create our own implementation just yet. I'll use an implementation provided by Spring Security, named `InMemoryUserDetailsManager`. With this example, you'll learn how to plug this kind of object into your architecture.

NOTE Interfaces in Java define contracts between objects. In the class design of the application, we use interfaces to decouple objects that use one another. To enforce this interface characteristic when discussing those in this book, I mainly refer to them as *contracts*.

To show you how to override this component with an implementation we chose, we'll change what we did in the first example. Doing so allows us to have our own managed credentials for authentication. For this example, we don't implement our class, but we use an implementation provided by Spring Security.

In this example, we use the `InMemoryUserDetailsManager` implementation. Even if it is a bit more than just a `UserDetailsService`, for now, we only refer to it from the perspective of a `UserDetailsService`. This implementation stores credentials in memory, which can then be used by Spring Security to authenticate a request.

NOTE An `InMemoryUserDetailsManager` implementation isn't meant for production-ready applications, but it's an excellent tool for examples or proofs of concept. In some cases, all you need are users. You don't need to spend the time implementing this part of the functionality. In our case, we use it to understand how to override the default `UserDetailsService` implementation.

We start by defining a configuration class. Generally, we declare configuration classes in a separate package named config. The following listing shows the definition for the configuration class. You can also find the example in the project ssia-ch2-ex2.

Listing 2.3 The configuration class for the `UserDetailsService` bean

```
@Configuration                ①
public class ProjectConfig {

  @Bean                       ②
  UserDetailsService userDetailsService() {
    return new InMemoryUserDetailsManager();
  }
}
```

① The @Configuration annotation marks the class as a configuration class.

② The @Bean annotation instructs Spring to add the returned value as a bean in the Spring context.

We annotate the class with `@Configuration`. The `@Bean` annotation instructs Spring to add the instance returned by the method to the Spring context. If you execute the code exactly as it is now, you'll no longer see the autogenerated password in the console. The application now uses the instance of type `UserDetailsService` you added to the context instead of the default autoconfigured one. But at the same time, you won't be able to access the endpoint anymore for two reasons:

- You don't have any users.
- You don't have a `PasswordEncoder`.

In figure 2.2, you saw that authentication depends on a `PasswordEncoder` as well. Let's solve these two problems step by step. We need to

1. Create at least one user who has a set of credentials (username and password)
2. Add the user to be managed by our implementation of `UserDetailsService`
3. Define a bean of the type `PasswordEncoder` that our application can use to verify a given password with the one stored and managed by `UserDetailsService`

First, we declare and add a set of credentials that we can use for authentication to the instance of `InMemoryUserDetailsManager`. In chapter 3, we'll discuss more about users and how to manage them. For the moment, let's use a predefined builder to create an object of the type `UserDetails`.

NOTE You'll sometimes see that I use `var` in the code. Java 10 introduced the reserved type name `var`, and you can only use it for local declarations. Although in some cases the way `var` is used in this book could be considered a bad approach from a clean coding perspective, it is done to make the syntax shorter, as well as to hide the variable type. This approach helps you focus on what is relevant for the given example. We'll discuss the types hidden by `var` in later chapters, so you don't have to worry about that type until it's time to analyze it properly.

When building the instance, we must provide the username, the password, and at least one authority. The *authority* is an action allowed for that user, and we can use any string for this. In the next listing, I name the authority `read`, but because we won't use this authority for the moment, this name doesn't really matter.

Listing 2.4 Creating a user with the `User` builder class for `UserDetailsService`

```
@Configuration
public class ProjectConfig {

  @Bean
  UserDetailsService userDetailsService() {
    var user = User.withUsername("john")          ①
          .password("12345")                      ①
          .authorities("read")                    ①
          .build();                               ①

    return new InMemoryUserDetailsManager(user);  ②
  }
}
```

① Builds the user with a given username, password, and authorities list

② Adds the user to be managed by UserDetailsService

NOTE You'll find the class `User` in the org.springframework.security.core.userdetails package. It's the builder implementation we use to create the object to represent the user. Furthermore, as a general rule in this book, if I don't present how to write a class in a code listing, it means Spring Security provides it.

As presented in listing 2.4, we must provide a value for the username, one for the password, and at least one authority. However, this is still not enough to allow us to call the endpoint. We also need to declare a `PasswordEncoder`.

When using the default `UserDetailsService`, a `PasswordEncoder` is also autoconfigured. Because we overrode `UserDetailsService`, we also have to declare a `PasswordEncoder`. Trying the example now, you'll see an excep-

tion when you call the endpoint. When trying to do the authentication, Spring Security realizes it doesn't know how to manage the password and fails. The exception looks like that in the next code snippet, and you should see it in your application's console. The client gets back an HTTP 401 Unauthorized message and an empty response body:

```
curl -u john:12345 http://localhost:8080/hello
```

The result of the call in the app's console is

```
java.lang.IllegalArgumentException:
There is no PasswordEncoder mapped for the id "null"
    at
org.springframework.security.crypto.
➥password.DelegatingPasswordEncoder$
➥UnmappedIdPasswordEncoder.matches(
➥DelegatingPasswordEncoder.java:289)
➥~[spring-security-crypto-6.0.0.jar:6.0.0]
    at org.springframework.security.crypto.
➥password.DelegatingPasswordEncoder.matches(
➥DelegatingPasswordEncoder.java:237)
➥~[spring-security-crypto-6.0.0.jar:6.0.0]
```

To solve this problem, we can add a `PasswordEncoder` bean in the context, like we did with the `UserDetailsService`. For this bean, we use an existing implementation of `PasswordEncoder`:

```
@Bean
public PasswordEncoder passwordEncoder() {
 return NoOpPasswordEncoder.getInstance();
}
```

NOTE The `NoOpPasswordEncoder` instance treats passwords as plain text. It doesn't encrypt or hash them. For matching, `NoOpPasswordEncoder` only compares the strings using the underlying `equals(Object o)` method of the `String` class. You shouldn't use this type of `PasswordEncoder` in a production-ready app. `NoOpPasswordEncoder` is a good option for examples where you don't want to focus on the hashing algorithm of the password. Therefore, the developers of the class marked it as `@Deprecated`, and your development environment will show its name with a strikethrough.

You can see the full code of the configuration class in the following listing.

Listing 2.5 The full definition of the configuration class

```
@Configuration
public class ProjectConfig {

 @Bean
 UserDetailsService userDetailsService() {
  var user = User.withUsername("john")
          .password("12345")
          .authorities("read")
          .build();
```

```
    return new InMemoryUserDetailsManager(user);
  }

  @Bean                          ①
  PasswordEncoder passwordEncoder() {
    return NoOpPasswordEncoder.getInstance();
  }
}
```

① A new method annotated with @Bean to add a PasswordEncoder to the context

Let's try the endpoint with the new user having the username `john` and the password `12345`:

```
curl -u john:12345 http://localhost:8080/hello
Hello!
```

NOTE Knowing the importance of unit and integration tests, some of you might already wonder why we don't also write tests for our examples. You will actually find the related Spring Security integration tests with all the examples provided in this book. However, to help you focus on the presented topics for each chapter, I have separated the discussion about testing Spring Security integrations and detail this in chapter 18.

2.3.2 Applying authorization at the endpoint level

With new management for the users in place, as described in section 2.3.1, we can now discuss the authentication method and configuration for endpoints. You'll learn plenty of things regarding authorization configuration in chapters 7 through 12. But before diving into details, you must understand the big picture. The best way to achieve this is with our first example. With the default configuration, all the endpoints assume you have a valid user managed by the application. Also, by default, your app uses HTTP Basic authentication, but you can easily override this configuration.

As you'll learn in the next chapters, HTTP Basic authentication doesn't fit into most application architectures. Sometimes, we'd like to change it to match our application. Similarly, not all endpoints of an application need to be secured, and for those that do, we might need to choose different authentication methods and authorization rules. To customize the handling of authentication and authorization, we'll need to define a bean of type `SecurityFilterChain`. For this example, I'll continue writing the code in the project ssia-ch2-ex3.

Listing 2.6 Defining a `SecurityFilterChain` bean

```
@Configuration
public class ProjectConfig {

  @Bean
  SecurityFilterChain configure(HttpSecurity http)
    throws Exception {

    return http.build();
  }

  // Omitted code
}
```

We can then alter the configuration using different methods of the `HttpSecurity` object, as shown in the next listing.

Listing 2.7 Using the `HttpSecurity` parameter to alter the configuration

```
@Configuration
public class ProjectConfig {

  @Bean
  SecurityFilterChain configure(HttpSecurity http)
    throws Exception {

    http.httpBasic(Customizer.withDefaults());      ①

    http.authorizeHttpRequests(
      c -> c.anyRequest().authenticated()           ②
    );

    return http.build();
  }

  // Omitted code

}
```

① App uses HTTP Basic authentication.

② All the requests require authentication.

The code in listing 2.7 configures endpoint authorization with the same behavior as the default one. You can call the endpoint again to see whether it behaves the same as in the previous test from section 2.3.1. With a slight change, you can make all the endpoints accessible without the need for credentials. You'll see how to do this in the following listing.

Listing 2.8 Using `permitAll()` to change the authorization configuration

```
@Configuration
public class ProjectConfig {

  @Bean
  public SecurityFilterChain configure(HttpSecurity http)
    throws Exception {

    http.httpBasic(Customizer.withDefaults());

    http.authorizeHttpRequests(
      c -> c.anyRequest().permitAll()       ①
    );

    return http.build();
  }

  // Omitted code
}
```

① None of the requests need to be authenticated.

Now we can call the `/hello` endpoint without the need for credentials. The `permitAll()` call in the configuration, together with the `anyRequest()` method, makes all the endpoints accessible without the need for credentials:

```
curl http://localhost:8080/hello
```

The response body of the call is

```
Hello!
```

In this example, we used two configuration methods:

- `httpBasic()`, which helped us configure the authentication approach. By calling this method, you instructed your app to accept HTTP Basic as an authentication method.
- `authorizeHttpRequests()`, which helped us configure the authorization rules at the endpoint level. By calling this method, you instructed the app on how to authorize the requests received on specific endpoints.

For both methods, you had to use a `Customizer` object as a parameter. `Customizer` is a contract you implement to define the customization for either Spring Security element you configure: the authentication, the authorization, or particular protection mechanisms such as CSRF or CORS (which will be discussed in chapters 9 and 10). The following snippet shows the definition of the `Customizer` interface. Observe that `Customizer` is a functional interface (so we can use lambda expressions to implement it), and the `withDefaults()` method I used in listing 2.8 is, in fact, just a `Customizer` implementation that does nothing:

```
@FunctionalInterface
public interface Customizer<T> {
 void customize(T t);

 static <T> Customizer<T> withDefaults() {
  return (t) -> {
  };
 }
}
```

In earlier Spring Security versions, you could apply configurations without a `Customizer` object by using a chaining syntax, as shown in the following code snippet. Observe that instead of providing a `Customizer` object to the `authorizeHttpRequests()` method, the configuration just follows the method's call:

```
http.authorizeHttpRequests()
      .anyRequest().authenticated()
```

The reason this approach has been left behind is because a `Customizer` object allows you more flexibility in moving the configuration where needed. Sure, with simple examples, using lambda expressions is comfortable. But in real-world apps, the configurations can grow a lot. In such cases, the ability to move these configurations in separate classes helps you keep the configurations easier to maintain and test.

The purpose of this example is to give you a feeling for how to override default configurations. We'll get into the details about authorization in chapters 7 through 10.

NOTE In earlier versions of Spring Security, a security configuration class needed to extend a class named `WebSecurityConfigurerAdapter`. We don't use this practice anymore. If your app uses an older codebase, or you need to upgrade an older codebase, I recommend you also read the first edition of *Spring Security in Action*.

2.3.3 Configuring in different ways

One of the confusing aspects of creating configurations with Spring Security is having multiple ways to configure the same thing. In this section, you'll learn alternatives for configuring `UserDetailsService` and `PasswordEncoder`. It's essential to know the options you have so that you can recognize these in the examples that you find in this book or other sources like blogs and articles. It's also important that you understand how and when to use these in your application. Further chapters present different examples that extend the information in this section.

Let's take the first project. After we created a default application, we managed to override `UserDetailsService` and `PasswordEncoder` by adding new implementations as beans in the Spring context. Let's find another way of doing the same configurations for `UserDetailsService` and `PasswordEncoder`.

We can directly use the `SecurityFilterChain` bean to set both the `UserDetailsService` and the `PasswordEncoder`, as shown in the following listing. You can find this example in the project ssia-ch2-ex3.

Listing 2.9 Setting `UserDetailsService` with the `SecurityFilterChain` bean

```
@Configuration
public class ProjectConfig {

 @Bean
 public SecurityFilterChain configure(HttpSecurity http)
  throws Exception {

  http.httpBasic(Customizer.withDefaults());
  http.authorizeHttpRequests(
     c -> c.anyRequest().authenticated()
  );

  var user = User.withUsername("john")          ①
     .password("12345")
     .authorities("read")
     .build();

  var userDetailsService =                      ②
   new InMemoryUserDetailsManager(user);

  http.userDetailsService(userDetailsService);  ③

  return http.build();
}

 // Omitted code

}
```

① Defines a user with all its details

② Declares a UserDetailsSevice to store the users in memory and adds the user to be managed by our UserDetailsSevice

③ The UserDetailsService is now set up using the SecurityFilterChain bean.

In listing 2.9, you can observe that we declare the `UserDetailsService` in the same way as in listing 2.5. The difference is that this is now done locally inside the bean method creating the `SecurityFilterChain`. We also call the `userDetailsService()` method from the `HttpSecurity` to register the `UserDetailsService` instance. The next listing shows the full contents of the configuration class.

Listing 2.10 Full definition of the configuration class

```
@Configuration
public class ProjectConfig {

  @Bean
  SecurityFilterChain configure(HttpSecurity http)
    throws Exception {

    http.httpBasic(Customizer.withDefaults());

    http.authorizeHttpRequests(
       c -> c.anyRequest().authenticated()
    );

    var user = User.withUsername("john")          ①
       .password("12345")
       .authorities("read")
       .build();

    var userDetailsService =                      ②
       new InMemoryUserDetailsManager(user);

    http.userDetailsService(userDetailsService);  ③

    return http.build();
  }

  @Bean
  PasswordEncoder passwordEncoder() {
    return NoOpPasswordEncoder.getInstance();
  }
}
```

① Creates a new user

② Adds the user to be managed by our UserDetailsService

③ Configures UserDetailsService

Any of these configuration options are correct. The first option, where we add the beans to the context, lets you inject the values in another class where you might potentially need them. But if you don't need that for your case, the second option would be equally good.

2.3.4 Defining custom authentication logic

As you've already observed, Spring Security components provide a lot of flexibility, offering many options when adapting them to the architecture of our applications. Up to now, you've learned the purpose of `UserDetailsService` and `PasswordEncoder` in the Spring Security architecture. You also saw a few ways to configure them. It's time to learn how you can also customize the component that delegates to these, the `AuthenticationProvider`, as shown in figure 2.3. `AuthenticationProvider` implements the authentication logic and delegates to the `UserDetailsService` and `PasswordEncoder` for user and password management. Thus, we could say that with this section, we go one step deeper into the authentication architecture to learn how to implement custom authentication logic with `AuthenticationProvider`.

Because this is the first example, I only show you a brief picture so that you better understand the relationship between the components in the architecture. But we'll go into more detail in chapters 3 through 6.

I recommend you consider the responsibilities as designed in the Spring Security architecture. This architecture is loosely coupled with fine-grained responsibilities. That design is one of the things that makes Spring Security flexible and easy to integrate with your applications. Depending on how you make use of its flexibility, you could change the design as well. You must be careful with these approaches as they can complicate your solution. For example, you could choose to override the default `AuthenticationProvider` in a way in which you no longer needed a `UserDetailsService` or `PasswordEncoder`. With that in mind, listing 2.11 shows how to create a custom authentication provider. You can find this example in the project ssia-ch2-ex4.

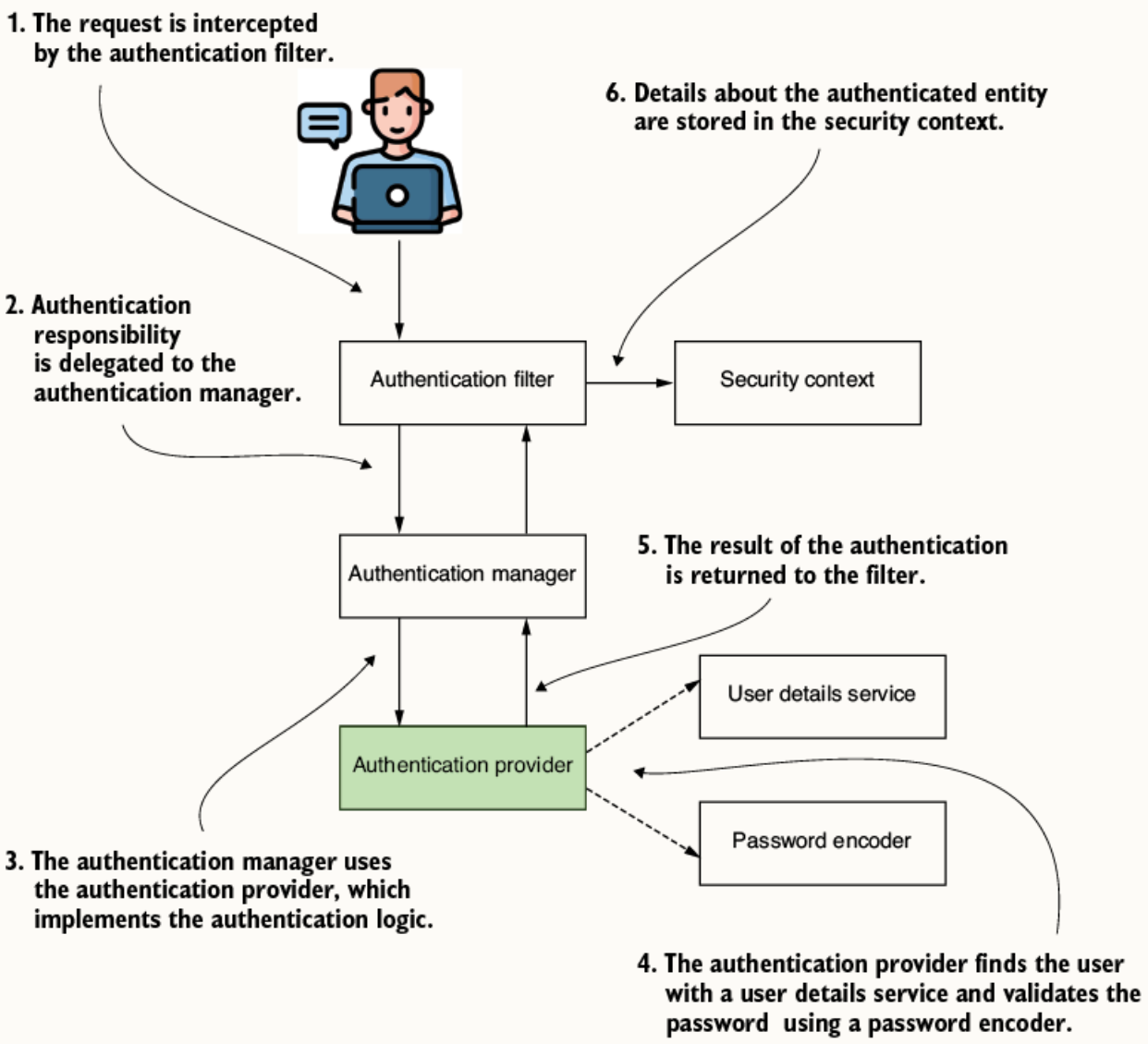


Figure 2.3 The `AuthenticationProvider` implements the authentication logic. It receives the request from the `AuthenticationManager` and delegates finding the user to a `UserDetailsService`, verifying the password to a `PasswordEncoder`.

Listing 2.11 Implementing the `AuthenticationProvider` interface

```
@Component
public class CustomAuthenticationProvider implements AuthenticationProvider {

 @Override
 public Authentication authenticate (Authentication authentication) throws AuthenticationException {

  // authentication logic here
 }

 @Override
 public boolean supports(Class<?> authenticationType) {

  // type of the Authentication implementation here
 }
}
```

The `authenticate(Authentication authentication)` method represents all the logic for authentication, so we'll add an implementation like that in listing 2.12. I'll explain the usage of the `supports()` method in detail in chapter

6. For the moment, I recommend you take its implementation for granted. It's not essential for the current example.

Listing 2.12 Implementing the authentication logic

```
@Override
public Authentication authenticate(
  Authentication authentication)
    throws AuthenticationException {

  String username = authentication.getName();      ①
  String password = String.valueOf(
    authentication.getCredentials());

  if ("john".equals(username) &&                   ②
      "12345".equals(password)) {
    return new UsernamePasswordAuthenticationToken(
               username,
               password,
               Arrays.asList());
  } else {
    throw new AuthenticationCredentialsNotFoundException("Error!");
  }

}
```

① The getName() method is inherited by Authentication from the Principal interface.

② This condition generally calls UserDetailsService and PasswordEncoder to test the username and password.

Here the condition of the `if-else` clause is replacing the responsibilities of `UserDetailsService` and `PasswordEncoder`. You are not required to use the two beans, but if you work with users and passwords for authentication, I strongly suggest you separate the logic of their management. Apply it as the Spring Security architecture designed it, even when you override the authentication implementation.

You might find it useful to replace the authentication logic by implementing your own `AuthenticationProvider`. If the default implementation doesn't fit entirely into your application's requirements, you can decide to implement custom authentication logic. The full `AuthenticationProvider` implementation looks like the one in the next listing.

Listing 2.13 The full implementation of the authentication provider

```
@Component
public class CustomAuthenticationProvider
  implements AuthenticationProvider {

  @Override
  public Authentication authenticate(
    Authentication authentication)
      throws AuthenticationException {

    String username = authentication.getName();
    String password = String.valueOf(authentication.getCredentials());
```

```
    if ("john".equals(username) &&
        "12345".equals(password)) {
      return new UsernamePasswordAuthenticationToken(
          username, password, Arrays.asList());
    } else {
      throw new AuthenticationCredentialsNotFoundException("Error!");
    }
  }

  @Override
  public boolean supports(Class<?> authenticationType) {
    return UsernamePasswordAuthenticationToken
        .class
        .isAssignableFrom(authenticationType);
  }
}
```

In the configuration class, you can register the `AuthenticationProvider` using the `HttpSecurity authentication-Provider()` method shown in the following listing.

Listing 2.14 Registering the new implementation of `AuthenticationProvider`

```
@Configuration
public class ProjectConfig {

 private final CustomAuthenticationProvider authenticationProvider;

 public ProjectConfig(
  CustomAuthenticationProvider authenticationProvider) {

  this.authenticationProvider = authenticationProvider;
 }

 @Bean
 SecurityFilterChain configure(HttpSecurity http) throws Exception {
  http.httpBasic(Customizer.withDefaults());

  http.authenticationProvider(authenticationProvider);

  http.authorizeHttpRequests(
   c -> c.anyRequest().authenticated()
  );

  return http.build();
 }
}
```

You can now call the endpoint, which is accessible by the only user recognized, as defined by the authentication logic `john`, with the password `12345`:

```
curl -u john:12345 http://localhost:8080/hello
```

The response body is

```
Hello!
```

In chapter 6, you'll learn more about the `AuthenticationProvider` and how to override its behavior in the authentication process. In the same chapter, we'll also discuss the `Authentication` interface and its implementations, such as `UserPasswordAuthenticationToken`.

2.3.5 Using multiple configuration classes

In the previously implemented examples, we only used a configuration class. It is, however, good practice to separate the responsibilities even for the configuration classes. We need this separation because the configuration starts to become more complex. In a production-ready application, you probably have more declarations than in our first examples. You also might find it useful to have more than one configuration class to make the project readable.

It's always a good practice to have only one class per responsibility. For this example, we can separate user management configuration from authorization configuration. We do that by defining two configuration classes: `UserManagementConfig` (defined in the next listing) and `WebAuthorizationConfig` (defined in listing 2.16). You can find this example in the project ssia-ch2-ex5.

Listing 2.15 Defining the configuration class for user and password management

```
@Configuration
public class UserManagementConfig {

 @Bean
 public UserDetailsService userDetailsService() {
  var userDetailsService = new InMemoryUserDetailsManager();

  var user = User.withUsername("john")
          .password("12345")
          .authorities("read")
          .build();

  userDetailsService.createUser(user);
  return userDetailsService;
 }

 @Bean
 public PasswordEncoder passwordEncoder() {
  return NoOpPasswordEncoder.getInstance();
 }
}
```

In this case, the `UserManagementConfig` class only contains the two beans responsible for user management: `UserDetailsService` and `PasswordEncoder`. The next listing shows this definition.

Listing 2.16 Defining the configuration class for authorization management

```
@Configuration
public class WebAuthorizationConfig {

 @Bean
```

```
  SecurityFilterChain configure(HttpSecurity http)
    throws Exception {

    http.httpBasic(Customizer.withDefaults());

    http.authorizeHttpRequests(
       c -> c.anyRequest().authenticated()
    );

    return http.build();
  }
}
```

Here the `WebAuthorizationConfig` class needs to define a bean of type `SecurityFilterChain` to configure the authentication and authorization rules.

Summary

- Spring Boot provides some default configurations when you add Spring Security to the dependencies of the application.
- You implement the following basic components for authentication and authorization: `UserDetailsService`, `PasswordEncoder`, and `AuthenticationProvider`.
- You can define users with the `User` class. A user should at least have a username, a password, and an authority. Authorities are actions that you allow a user to do in the context of the application.
- A simple implementation of a `UserDetailsService` that Spring Security provides is `InMemoryUserDetailsManager`. You can add users to such an instance of `UserDetailsService` to manage the user in the application's memory.
- The `NoOpPasswordEncoder` is an implementation of the `PasswordEncoder` contract that uses passwords in cleartext. This implementation is good for learning examples and (maybe) proofs of concept, but not for production-ready applications.
- You can use the `AuthenticationProvider` contract to implement custom authentication logic in the application.
- There are multiple ways to write configurations, but in a single application, you should choose and stick to one approach. This helps to make your code cleaner and easier to understand.

Previous chapter
< 1 Security today

Next chapter
Part 2. Configuring authentication >